

1 P4 C4: range proof and exact method audit

Task source: [Every k up to 12](#), checked on 2026-09-26. This is a complete local mathematical and computational solution; no platform submission or organizer acceptance is claimed.

1.1 Claim and meaning of the computation

For every integer k with $2 \leq k \leq 12$, every k pairwise disjoint integer congruence classes with positive moduli contain a pair whose moduli have gcd at least k . Repeated moduli are allowed.

A complete Lean proof is `ThroughTwelve.solution` in [ThroughTwelve.lean](#). Its theorem has no search-completion or smaller-case premises. The supplied `Statement` uses actual disjointness of integer sets. Bézout/CRT, normalization, the density inequality, enumeration correctness, and the concrete $k=11$ and $k=12$ refutations are proved inside the package. Lean 4.34.1 plus bundled `Std` suffices. Both a fresh build and a separate kernel replay passed; [verification record](#) gives commands and axiom audit.

For the requested audit at 9–16, distinguish two stages:

1. **Necessary modulus tests:** enumerate finite normalized modulus multisets, apply every-subset density and the explicitly described extra minimal-counterexample conditions.
2. **Actual residue decision:** decide every remaining multiset by complete finite residue search, with independently checked refutation trees and concrete witnesses proving minimum obstruction cardinalities.

The mathematical reduction and each pruning rule are proved below. The [audit manifest](#) locates the exact data and code. The residue tests examine all assignments after a proved finite normalization.

1.2 Written base cases: every size from 2 through 8

The following argument is reproduced from the independently verified C3 written proof. It establishes each lower case used by the least-counterexample reduction below; no lower-case hypothesis is left open.

Let $A_i = a_i \pmod{m_i}$ be k pairwise disjoint congruence classes of integers, where every m_i is a positive integer. Repeated moduli are permitted. We claim that some two of their moduli satisfy

$$\gcd(m_i, m_j) \geq k.$$

The elementary intersection criterion is

$$(a_i \pmod{m_i}) \cap (a_j \pmod{m_j}) \neq \emptyset \iff \gcd(m_i, m_j) \mid (a_j - a_i). \quad (\text{B1})$$

For the reverse implication, Bézout gives integers u, v with $g = um_i + vm_j$ for $g = \gcd(m_i, m_j)$. If $a_j - a_i = tg$, then $z = a_i + tum_i = a_j - tvm_j$ lies in both classes. The forward implication follows because an integer in both classes makes $a_j - a_i$ an integer linear combination of m_i, m_j . Thus disjointness says that a_i and a_j are *different modulo their modulus gcd*. We will use the following consequence: if $g_{ij} = \gcd(m_i, m_j)$ divides D , then $a_i \not\equiv a_j \pmod{D}$; equality modulo D would imply equality modulo g_{ij} .

Suppose, toward a contradiction for any fixed k , that every pair gcd is less than k . It cannot be 1, by (B1), so throughout that case

$$2 \leq g_{ij} < k \quad (i \neq j). \quad (\text{B2})$$

1.2.1 The cases $k = 2, 3, 4$

For $k = 2$, the sole gcd cannot be 1, proving the claim. For $k = 3$, (B2) makes every pair gcd equal to 2. Three residues would then be pairwise different modulo 2, an impossibility.

For $k = 4$, every pair gcd is 2 or 3. If all moduli are even, each pair gcd is 2, so already three residues would be distinct modulo 2. Otherwise choose an odd modulus. Its gcd with each other modulus must be 3, making every modulus divisible by 3. All pair gcds are then 3, forcing four distinct residues modulo 3. Both alternatives are impossible.

1.2.2 The case $k = 5$

Here every pair gcd lies in $\{2, 3, 4\}$. If all five moduli are even, each pair gcd is 2 or 4 and hence divides 4. The five residues must be pairwise distinct modulo 4, impossible.

Otherwise choose an odd modulus m_i . Its gcd with any other modulus is odd and in $\{2, 3, 4\}$, so equals 3. Consequently all five moduli are divisible by 3. Every pair gcd is now divisible by 3 and in $\{2, 3, 4\}$, so equals 3. Five pairwise distinct residues modulo 3 are impossible.

1.2.3 The case $k = 6$

Every pair gcd is in $\{2, 3, 4, 5\}$, so any two moduli share at least one prime from $P = \{2, 3, 5\}$. None of these three primes divides *all* six moduli. If 2 did, every pair gcd would be 2 or 4, forcing six pairwise distinct residues modulo 4. If 3 did, every pair gcd would equal 3, forcing six distinct residues modulo 3. If 5 did, every pair gcd would equal 5, forcing six distinct residues modulo 5. Each conclusion is impossible.

Every modulus is divisible by at least two primes from P . Indeed, it is divisible by at least one because it shares a prime from P with every other modulus. If it were divisible by just one, say p , every other modulus would have to share p with it, making p common to all six.

There are only three two-prime types, $\{2, 3\}$, $\{2, 5\}$, and $\{3, 5\}$. If two moduli contained the same type, their gcd would be at least the product of its primes, hence at least 6, contradicting (B2). Assign to each modulus any one of its contained two-prime types. The assignment is injective, so six moduli cannot be assigned to three types.

1.2.4 The case $k = 7$

Now every pair gcd is in $\{2, 3, 4, 5, 6\}$, and every pair shares at least one prime in $P = \{2, 3, 5\}$. Again no prime in P divides all seven moduli. If 3 did, each gcd would be 3 or 6 and hence divide 6; seven residues could not be pairwise distinct modulo 6. If 5 did, each gcd would equal 5; already six residues could not be pairwise distinct modulo 5.

If 2 divided all seven moduli, among the seven residues at least four would have the same parity $r \in \{0, 1\}$. Restrict to those four classes and write their residues and moduli as $a_s = 2b_s + r$ and $m_s = 2n_s$. The four reduced classes $b_s \pmod{n_s}$ are pairwise disjoint: an integer common to two reduced classes would give, after multiplying by 2 and adding r , an integer common to

the corresponding original classes. The established $k = 4$ case therefore supplies a pair with $\gcd(n_s, n_t) \geq 4$. Their original gcd is $2 \gcd(n_s, n_t) \geq 8$, contrary to (B2). Hence 2 is not common either.

As at $k = 6$, each modulus must contain at least two primes from P : it contains one because it shares a prime from P with each other modulus, and containing only one would make that prime common to all seven. Choose an index i for which $2 \nmid m_i$ and an index j for which $3 \nmid m_j$. Then $3 \cdot 5 \mid m_i$ and $2 \cdot 5 \mid m_j$, so $i \neq j$.

Consider any of the other five indices t . If $5 \mid m_t$, it also contains 2 or 3. In the first case $10 \mid \gcd(m_t, m_j)$; in the second $15 \mid \gcd(m_t, m_i)$. Either contradicts (B2). Therefore $5 \nmid m_t$, and the two-prime property forces $2 \cdot 3 \mid m_t$.

For distinct indices s, t among those five, $6 \mid \gcd(m_s, m_t)$, so (B2) makes their gcd exactly 6. For each such t , the gcd of m_t and m_i is divisible by 3, is not divisible by 2 because m_i is odd, and is at most 6; it is therefore exactly 3. Their five residues must consequently be pairwise distinct modulo 6 while all avoiding $a_i \pmod{3}$. Of the six classes modulo 6, exactly four avoid one prescribed class modulo 3. Five distinct choices from four are impossible.

1.2.5 The case $k = 8$

Here every pair gcd belongs to $\{2, 3, 4, 5, 6, 7\}$. Remove any one index. The remaining seven classes are still disjoint, so the proved $k = 7$ case gives a pair whose gcd is at least 7. By (B2), that gcd is exactly 7. Hence, for every removed index, two of the retained moduli are divisible by 7.

Let S be the set of indices whose moduli are divisible by 7. The preceding conclusion gives $|S| \geq 2$; in fact $|S| \geq 3$, because if it had exactly two members, removing either one would leave no possible pair with gcd 7. On the other hand, S is not all eight indices. If every modulus were divisible by 7, (B2) would make every pair gcd equal to 7, forcing eight distinct residues modulo 7.

Choose an index $t \notin S$. For every $s \in S$, the gcd of m_s and m_t is in $\{2, 3, 4, 5, 6\}$. It is therefore divisible by one of 2, 3, 5, which divides both moduli. Distinct $s_1, s_2 \in S$ cannot share any of these three primes, because 7 and that prime would both divide $\gcd(m_{s_1}, m_{s_2})$, making it at least 14, contrary to (B2). Thus each member of S requires a *different* prime from $\{2, 3, 5\}$ to intersect m_t in this divisibility sense. It follows that $|S| \leq 3$, hence $|S| = 3$.

There are therefore five indices outside S . The same argument applies to *every* outside index: its modulus is divisible by all three primes 2, 3, and 5, because it must share one distinct such prime with each of the three moduli in S . Any two outside moduli then have gcd at least $2 \cdot 3 \cdot 5 = 30$, contradicting (B2). This completes the proof through eight.

The bound is sharp at each k : the k classes $0, 1, \dots, k-1 \pmod{k}$ are pairwise disjoint, and every pair of moduli has gcd exactly k .

The [complete formal C3 proof](#) independently verifies this base range. Its full local Lean dependency closure is printed in the separate C3 cell; this C4 document keeps the written proof and the C4 computation together.

1.3 Finite reduction and pruning proof

This note proves the mathematical implications used by [enumerate.py](#), [clique.py](#), and the modulus and residue checks in [audit.py](#). The source is Kevin O'Bryant, *On Z.-W. Sun's Disjoint Congruence Classes Conjecture*, [arXiv:math/0604347v2](#), especially Proposition 2 and Lemmas

5–6. The arguments are reconstructed below rather than delegated to that citation. They apply to a **least-size** counterexample, and the sum-minimal normalization applies after choosing, among counterexamples of that size, one minimizing the sum of the moduli. They do not by themselves prove a conjecture at a size whose smaller cases are unsettled. This document establishes pruning soundness; completeness of a particular finite run additionally requires its recorded **complete: true** result and replay of its output.

1.3.1 Starting hypothesis and finite normalization

Assume $k \geq 4$ is the least size of pairwise disjoint integer congruence classes $a_i \pmod{m_i}$, with positive moduli, for which every $\gcd(m_i, m_j) < k$. Choose such a system with minimal $\sum_i m_i$. By the two-class Chinese remainder theorem, classes $a_i \pmod{m_i}$ and $a_j \pmod{m_j}$ meet if and only if $\gcd(m_i, m_j) \mid a_i - a_j$. Thus all pair gcds are in $\{2, \dots, k-1\}$; a gcd of 1 cannot occur in a disjoint system. This is the paper’s [Proposition 2](#).

Let

$$N = \text{lcm}_{i < j} \gcd(m_i, m_j), \quad L_k = \text{lcm}(1, 2, \dots, k-1).$$

Certainly $N \mid L_k$. If a maximal prime-power factor p^e of some m_i did not divide N , every other m_j would have p -adic valuation below e : otherwise $p^e \mid \gcd(m_i, m_j) \mid N$. Dividing just m_i by p consequently leaves every $\gcd(m_i, m_j)$ unchanged, since its p -valuation was already less than e . All disjointness inequalities remain true by the Chinese remainder criterion, while the modulus sum decreases. This contradicts its choice. Every maximal prime-power factor of every m_i therefore divides N , so $m_i \mid N \mid L_k$. Also $N = \text{lcm}_i m_i$: every pair gcd divides its participating moduli, and every modulus divides N . This proves the finite reduction in [Lemma 6\(1\)](#), pp. 3–4.

Every prime divisor of any m_i is therefore a prime below k . A modulus 1 is impossible because all its pair gcds would be 1. No modulus is a prime power. To prove the latter, suppose $m_i = p^e$. Every modulus is divisible by p , since it has gcd greater than 1 with m_i ; also $p < k$. Among the k residues, choose $t = \lceil k/p \rceil$ with one common residue modulo p , where $2 \leq t < k$. Subtract that common residue and divide these selected residues and moduli by p . For every selected pair, both its residue difference and pair gcd are divided by p , hence the divided classes remain disjoint. Minimality of k forces a pair gcd at least t among them, making the original pair gcd at least $pt \geq k$, a contradiction. This proves [Lemma 6\(4\)](#), pp. 4–5. Thus a complete finite *candidate* domain for modulus **values** is the set of divisors of L_k having at least two distinct prime factors; later conditions may exclude individual values or combinations. Both `enumerate.domain` (prime-power product expansion) and `clique.divisors_with_two_primes` (divisor generation and trial division) compute that set; repetitions remain possible.

For every prime power $q \mid m_i$, also $q \mid N$. Since N is the lcm of pair gcds, some pair gcd is divisible by q , so at least two moduli are divisible by q . Whether or not i belongs to that pair, at least one of those two moduli is different from i . This proves the *prime-power support* rule, [Lemma 6\(3\)](#). The code tests the maximal power of each prime in each modulus; testing these powers suffices because every lower power is then supported as well. `enumerate.power_support`, `audit.maximal_power_failure`, and `clique.extra_good` all implement this implication on complete multisets.

1.3.2 Anchor counts

Call a modulus an anchor when $k-1 \mid m_i$. There must be at least three anchors. If there were at most two, delete one anchor, or any class if there are none. The remaining $k-1$ classes would

be disjoint and no pair gcd could equal $k - 1$, since that would require two remaining anchors. Every pair gcd would be below $k - 1$, contradicting the least-size choice. This is [Lemma 6\(5\)](#), and justifies beginning both searches with at least three anchors.

If there are **exactly** three anchors, there must be at least two *other* moduli divisible by $d = k - 2$. Here is a direct proof of [Lemma 6\(6\)](#). Retain any one anchor and delete the other two. The remaining $k - 2$ disjoint classes cannot have all pair gcds below $k - 2$; otherwise they form a smaller counterexample. Their gcds are still below k , and a gcd of $k - 1$ is impossible with only one anchor remaining. Hence some two remaining moduli are divisible by d . If no outside modulus is divisible by d , this is impossible. If exactly one outside modulus x is divisible by d , repeating the deletion with each of the three possible retained anchors shows that all three anchors are divisible by d . Two anchors would then have gcd divisible by $\text{lcm}(k - 1, k - 2) = (k - 1)(k - 2) \geq k$, impossible. Thus at least two outside moduli are d -divisible. `special_obstruction` and `extra_good` check exactly that outside count when the final anchor count is three.

1.3.3 The large-prime rule, including its exceptional case

For $7 \leq k \leq 30$, let $p \geq k/2$ be a prime dividing a modulus. Prime-power support gives at least two p -divisible moduli, and pair-gcd bounds imply $p < k$. Suppose there are $\ell \geq 3$ of them. For each such modulus m_i , let P_i contain its prime divisors other than p . Each P_i is nonempty by the prime-power exclusion. They are pairwise disjoint: a prime q in two of them would give a pair gcd divisible by $pq \geq 2p \geq k$. All these primes are below k by finite normalization.

Each of the other $k - \ell$ moduli meets every anchor in gcd greater than 1. It is not divisible by p , so it shares some prime in each P_i . Choosing one shared prime per anchor gives a signature in $P_1 \times \cdots \times P_\ell$. Equal signatures for two outside moduli would make their gcd divisible by the product of ℓ distinct primes. Because $\ell \geq 3$, this product is at least $2 \cdot 3 \cdot 5 = 30 \geq k$. The signature map is therefore injective, so

$$k - \ell \leq \prod_{i=1}^{\ell} |P_i|. \quad (1)$$

If r is the number of primes below k , then $\sum_i |P_i| \leq r - 1$: p is omitted and the P_i are disjoint. For a fixed positive-integer sum, the product is maximal when sizes differ by at most one; moving one unit from a to b when $a \geq b + 2$ raises the product by $a - b - 1$. Using all $r - 1$ available units gives the upper bounds below. A dash means $\ell > r - 1$, already impossible because each support is nonempty.

r	smallest possible k in $7 \leq k \leq 30$	$\ell = 3$	4	5	6	7	8
3	7	—	—	—	—	—	—
4	8	1	—	—	—	—	—
5	12	2	1	—	—	—	—
6	14	4	2	1	—	—	—
7	18	8	4	2	1	—	—
8	20	12	8	4	2	1	—
9	24	18	16	8	4	2	1
10	30	27	24	16	8	4	2

For rows $r = 3, \dots, 9$, each numeric bound is strictly below $k - \ell$, even using the row's smallest possible k . For $r = 10$, $k = 30$; every entry with $\ell \geq 4$ is below $30 - \ell$. The one equality is $k = 30, \ell = 3$, where (1) forces all three P_i to have size 3, to partition all nine primes other than p , and to realize all 27 signatures on the 27 outside moduli. Of the four primes 17, 19, 23, 29, precisely one can be p , so some $q \geq 17$ belongs to one support. Choose a prime q' in another support; $qq' \geq 34 > 30$. Fix the signature coordinates q, q' . The third coordinate has three choices, each realized by a distinct outside modulus. Any two of those moduli have gcd at least $qq' > 30$, a contradiction. Hence ℓ cannot exceed 2, and support forces $\ell = 2$. This is [Lemma 6\(8\)](#), pp. 5–6, with its final exceptional-case inference made explicit.

The paper's final sentence says two of 17, 19, 23, 29 must lie in separate P_i . That statement does not follow literally when the excluded prime p is one of the four: the other three could occupy one size-three support. The q, q' argument above covers this configuration. `enumerate.py` applies the rule only for $7 \leq k \leq 30$. `clique.py` currently tests `k <= 30` without the lower bound; its published runs begin at $k = 9$, so they are inside the proved range. Its generic strong-mode pruning for $k < 7$ is not justified by this argument.

1.3.4 Density for every position subset

For any subset S of at least two *positions*, define $M_S = \text{lcm}_{i < j \in S} \text{gcd}(m_i, m_j)$. Modulo M_S , the class $a_i \pmod{m_i}$ occupies exactly $M_S / \text{gcd}(m_i, M_S)$ residue classes. These occupied sets for distinct $i \in S$ must be disjoint: a common residue modulo M_S would make $a_i - a_j$ divisible by $\text{gcd}(m_i, m_j)$, since that gcd divides M_S , and the original two classes would meet. Counting the M_S available residues gives

$$\sum_{i \in S} \frac{M_S}{\text{gcd}(m_i, M_S)} \leq M_S. \quad (2)$$

Thus an exact integer total greater than M_S rejects the tuple, as in [Lemma 5](#) and [Lemma 6\(7\)](#), pp. 2–3. If M is any multiple of M_S , then $\text{gcd}(m_i, M_S) \mid \text{gcd}(m_i, M)$, so each reciprocal term for M is no larger than for M_S . Testing M_S therefore covers every larger permitted period. For a pair, (2) follows automatically from pair gcd at least 2: its two terms total $2/M_S \leq 1$. It suffices for the scripts to enumerate subset sizes at least three. Repeated values still represent different positions, so their multiplicities are retained in the sum.

`density_obstruction`, `density_bad`, and `direct_density_witness` compute (2) with integer `lcm`, `gcd`, and division. A bad prefix, triple, or other selected position subset stays in every extension, making each early density rejection hereditary. In `clique.py`, once c copies of a value form a bad prefix or triple, increasing c cannot repair that already-present subset; the loop's `break` is sound. At a leaf, `first_bad_subset` and `all_subsets_good` enumerate every position subset of size at least three. In particular, rejection does not depend on a floating-point threshold or on a density heuristic.

1.3.5 Search-path and partial-pool coverage

`enumerate.py` constructs every nondecreasing anchor triple with replacement. For any eligible k -multiset, its first three anchors give exactly one such triple; the remaining occurrences can then be appended in nondecreasing order from `pool`. The initial pool contains every nonanchor compatible with the triple and every compatible anchor no smaller than its third value, **including equal values**. At each extension, `following = pool[i:]` preserves exactly the allowed nondecreasing suffix, intersected with compatibility with the newly chosen value. Thus reject-

ing `gcd <= 1` or `gcd >= k` at the triple or extension level removes only impossible pairs, and every eligible multiset has a search path. The `--target-length` option is used for diagnostics; strong minimal-counterexample pruning is explicitly forbidden by the code when `target_length != k`.

`clique.py` independently enumerates distinct compatible modulus values, gives each a positive multiplicity, and sorts the result. A repeated value $x \geq k$ is impossible because its pair gcd would be $x \geq k$. A repeated value $x < k$ has multiplicity at most x : for c identical classes, (2) with $M_S = x$ says $c/x \leq 1$. These are the `max_count` bounds. Candidate distinct values form a clique under pair compatibility. Selecting all anchor values first and then all nonanchor values loses no multiset, since order of modulus positions has no mathematical effect. At the anchor stage, `possible_others` contains every nonanchor compatible with the chosen anchors; at the other stage, each recursive `allowed` suffix contains every later distinct compatible value. The capacity rejection

`len(xs) + sum(min(k,x) if x < k else 1 for x in candidates) < k`

uses an **upper** bound on how many positions the remaining values can supply. If even that bound cannot reach k , no completion exists. The analogous anchor-stage inequality with right side 3 only rejects states unable to reach the required three anchors. `expand_others` is called at each anchor state of length at least three, so all possible final anchor counts are covered. Its and `expand_anchors`'s density and triple **breaks** have the hereditary justification above.

Both strong modes use an overapproximation of future choices, which makes their three no-future rejections safe:

1. **Prime-power support.** If a maximal power q now appears once and no value in the future pool is divisible by q , it can never attain the required second occurrence.
2. **Large-prime count.** For $7 \leq k \leq 30$, more than two current multiples of $p \geq k/2$ cannot be removed. Exactly one current multiple with no future p -multiple cannot reach the required two. (`clique.py` has the lower-bound caveat noted above.)
3. **Exactly three anchors.** If there are exactly three current anchors, no future anchor, fewer than two current **outside** $k - 2$ -multiples, and no future $k - 2$ -multiple, the final tuple violates the proved exact-three rule.

In `enumerate.py`, `pool` is the complete sorted suffix of compatible future values. In `clique.py`, `candidates` is the complete suffix of distinct future values in `expand_others`; at an anchor state, `candidates + possible_others` is a possibly larger-than-real set of future values. Missing support from these pools therefore implies missing support from every actual continuation. The strong partial rules need no estimate of remaining positions; overlooking an impossible branch is harmless, while every branch they reject violates a necessary condition. At complete tuples, `special_obstruction` and `extra_good` apply the same three conditions directly.

1.3.6 Residue-checking caveat and audit scope

For a fixed modulus tuple, pairwise disjointness depends only on residues modulo $R_i = \text{lcm}_{j \neq i} \text{gcd}(m_i, m_j)$, and $R_i \mid m_i$. Common translation can set the first residue to zero; equal original moduli can be permuted so their residues increase with index. These are sound finite-domain reductions for an exact residue search. A branch is impossible if a proposed residue has pair-gcd divisibility with an assigned one, if an equal-original-modulus order is violated, or if an unassigned variable has no admissible residue. The separately exported trees in `export_refutations.py` and independently replayed trees in `verify_refutations.py` use the

equal-original-modulus condition `xs[j] == xs[i]` and check every admissible child and each dead leaf.

An earlier `audit.py` version compared `xs[j]` to the effective bound R_i , rather than to `xs[i]`. Our first audit called this unsound because the original moduli could differ. **That diagnosis was too strong and is withdrawn.** The old comparison also describes a valid compressed symmetry: if $m_j = R_i = q$, then $q \mid m_i$ and every $\gcd(m_i, m_\ell)$ divides q . For every other position ℓ , therefore,

$$\gcd(m_i, m_\ell) = \gcd(q, m_\ell) = \gcd(m_j, m_\ell).$$

Moreover $\gcd(m_i, m_j) = q$ and $R_j = q$. The two finite residue variables have identical bounds and identical constraints with all other variables. They may be exchanged even if their original moduli differ. Within each such effective-modulus group, residues are distinct modulo q and can be sorted by index; this satisfies all comparisons imposed by the old code. Thus we found a mismatch between the documented symmetry and its implementation, not a counterexample to search soundness.

The current code uses the simpler equal-original-modulus comparison, matching the proof above and the independent tree verifier. A fresh run from empty output files reproduced all 107 claims, all 20,436 fact keys and statuses, and every explicit SAT witness unchanged. The minimum verifier checked all 19,704 required smaller SAT facts; the tree verifier replayed all 59 refutations and 40,292 nodes. Both versions have sound symmetry reductions; the package relies on the current version and these independent certificates. Modulus enumeration never calls this residue solver.

1.4 Exact results

		Lists still undecided	
		Lists after extra modulus tests	after residue certificates
k	Raw density-only lists		
9	0	0	0
10	0	0	0
11	0	0	0
12	0	0	0
13	51	1	0
14	0	0	0
15	not certified in an unpruned run	30	0
16	not certified in an unpruned run	77	0

The chosen method at 15 and 16 includes extra necessary-condition pruning during enumeration. Its complete final survivor sets are the `reduced_survivors` arrays in [k=15](#) and [k=16](#). These files exhibit every modulus with multiplicity. For 9–14, the exact raw and reduced arrays are in [run-9-14.jsonl](#). A strong run’s intermediate `base_survivors` must not be described as the complete raw density-only set.

The second implementation chooses compatible distinct values and then their multiplicities, whereas the first grows occurrence lists from anchor triples. The recorded [elementwise com-](#)

[parison](#) has empty symmetric differences for every claimed final set. Fresh replay records node counts and wall clocks, and checks those counts against the baseline.

1.5 Why no surviving modulus list remains undecided

At $k=13$ the single reduced list is

`[10,10,10,10,10,12,12,12,12,24,36,40,45]`.

Its subset `[10,10,10,10,10,12,45]` is impossible. Disjointness from the 12-class forces all five 10-class residues to have the other parity. Pairwise disjointness among the 10-classes makes those five residues distinct modulo 10. They therefore exhaust all residues modulo 5. The 45-class meets one of them because its gcd with 10 is 5. This proves impossibility without computation. The 239 explicit assignments for every distinct nonempty submultiset of size at most six prove that seven is the smallest obstruction cardinality inside the full list. Both the impossibility and minimality are also Lean-certified in `Survivor13` and `Minimality13`.

At $k=15$ and 16 , [global-minima-claims.jsonl](#) gives each of the 107 lists, its individual obstruction, and its minimum cardinality. Every obstruction has a complete refutation tree; there are 59 distinct trees and 40,292 nodes. Every smaller submultiset has an explicit residue witness, checked directly by the CRT gcd criterion. Thus no smaller submultiset can force the negative decision. Repeated values are treated as separate positions when defining subfamilies, and duplicate submultisets share witnesses only because equal modulus multisets have exactly the same realizability question. The zero-entry subfamily is trivially realizable.

The smaller-case assumptions in the reduction are now closed explicitly. Suppose there were a counterexample at some size $2 \leq k \leq 16$, and choose the least such k ; among its systems, choose one with minimum modulus sum. The written base proof above excludes $k = 2, \dots, 8$. For each $k = 9, 10, 11, 12$, the complete reduced enumeration has no candidate modulus multiset, so the supposed minimum counterexample cannot exist. Thus the requested theorem holds through 12 even from the written finite certificate, independently of the separate Lean theorem. At $k = 13$ the sole candidate has the seven-class contradiction proved above. At $k = 14$ there is no candidate. At $k = 15$ and 16 , every one of the 30 and 77 complete reduced candidates contains a certified impossible submultiset. The least counterexample is impossible in every case. Hence the written computational certificate establishes the bound for all $2 \leq k \leq 16$; the independent Lean theorem formally proves the requested $2 \leq k \leq 12$ range. Each use of a smaller-size statement in the anchor and prime-power reductions therefore has a proved lower case behind it.

1.6 Source discrepancy, tested rather than assumed

O'Bryant's [paper](#), §4.4, reports no survivors through 19 from the displayed pair-gcd and every-subset density machinery. The explicit thirteen-list above passes those tests: every pair gcd is in $[2,12]$, and all 8,178 position subsets of cardinality at least two satisfy the canonical integer density inequality. Independent Python checking and `DensityAudit13.every_sublist_passes` establish this. Consequently that reported emptiness claim, interpreted as the stated necessary-condition tests, is incorrect. Our exhibited survivor is not a counterexample to the disjoint-congruence conjecture; its residue impossibility was proved above. We do not claim to have identified the cause of the paper's computational discrepancy.

During our own audit we found that the residue search used a stronger compressed-variable symmetry than the written explanation. We initially called this an error; further analysis shows the old variables were interchangeable because their effective bounds and gcd constraints

agree. That initial diagnosis is withdrawn, with proof in the [reduction document](#). The current implementation uses the simpler equal-original-modulus rule and reproduces every stored claim and witness unchanged. Independent saved-tree replay has always used that simpler rule.

1.7 Fresh exhaustive replay

`python3 -B cagent/p4/audit/replay.py --max-k 16` completed in **500.013 seconds**. Both searches reproduced their exact saved node counts and survivor lists at all eight sizes; their relevant lists agree elementwise. All certificate checks passed. The [evidence file](#) records per-size wall clocks, node counts, completion flags, positive control, and certificate results. The runner uses a new output directory by default.

k	Tuple recursion nodes	Graph recursion nodes	Tuple wall seconds	Graph wall seconds
9	10	20	0.031	0.03
10	92	85	0.032	0.033
11	9,541	7,460	0.107	0.106
12	773	699	0.042	0.036
13	347,628	217,024	5.2	3.795
14	12,884	8,359	0.149	0.081
15	2,330,980	1,522,808	59.622	57.823
16	6,921,594	4,111,538	201.727	165.504

The two algorithms count different recursive states, so their counts need not equal one another. Each exactly matches its own earlier run. The underlying JSONL records also include depth-specific counts and anchor-triple attempts where applicable.

1.8 Appendix: every surviving modulus list and its decision

The first entry is the sole reduced survivor at $k = 13$. The remaining 107 lines reproduce **every** reduced survivor at $k = 15, 16$, with multiplicities shown position by position. The arrow gives a complete submultiset that is impossible to realize by disjoint residues; **min** is its proven smallest cardinality within that survivor. Each 15–16 obstruction has an exhaustive refutation tree, and every smaller submultiset has a checked explicit residue assignment in the [claim/fact data](#) and [witness data](#). The $k = 13$ impossibility and all 239 smaller witnesses are proved or checked above. There are no reduced survivors at $k = 9, 10, 11, 12, 14$.

```

13#01 [10,10,10,10,10,12,12,12,12,24,36,40,45] -> [10,10,10,10,10,12,45]; min=7
15#01 [12,12,12,12,12,14,14,14,14,14,14,14,56,63,72] -> [12,12,12,12,12,14,63]; min=7
15#02 [12,12,12,12,12,14,14,14,14,14,14,14,21,56,70,120] -> [12,12,12,12,12,14,21]; min=7
15#03 [12,12,12,12,12,14,14,14,14,14,14,14,28,42,60,70] -> [12,12,12,12,14,14,14,14,14,14,28,70]; min=12
15#04 [12,12,12,12,12,14,14,14,14,14,14,14,28,63,70,180] -> [12,12,12,12,12,14,63]; min=7
15#05 [12,12,12,12,12,14,14,14,14,14,14,14,42,56,70,120] -> [12,12,12,12,14,14,14,14,14,14,56,70];
↪ min=12
15#06 [12,12,12,12,12,14,14,14,14,14,14,14,56,63,70,360] -> [12,12,12,12,12,14,63]; min=7
15#07 [12,12,12,12,14,14,14,14,14,14,14,14,24,36,56,63] -> [12,12,12,12,14,24,63]; min=7
15#08 [12,12,12,12,14,14,14,14,14,14,14,14,21,24,56,60,70] -> [12,12,12,12,14,21,24]; min=7
15#09 [12,12,12,12,14,14,14,14,14,14,14,14,24,42,56,60,70] -> [12,12,12,12,14,14,14,14,14,14,56,70]; min=12
15#10 [12,12,12,12,14,14,14,14,14,14,14,14,24,56,63,70,180] -> [12,12,12,12,14,24,63]; min=7
15#11 [12,12,12,12,14,14,14,14,14,14,14,14,28,36,60,63,70] -> [12,12,12,12,14,60,63]; min=7
15#12 [12,12,12,12,14,14,14,14,14,14,14,14,36,56,63,70,120] -> [12,12,12,12,14,63,120]; min=7
15#13 [12,12,12,12,14,14,14,14,14,14,14,14,56,60,63,70,72] -> [12,12,12,12,14,60,63]; min=7

```

15#14 [12,12,12,14,14,14,14,14,14,24,28,30,40,42,70] -> [12,12,12,14,30,40]; min=6
 15#15 [12,12,12,14,14,14,14,14,14,24,28,70,90,440,693] -> [12,12,12,14,90,440]; min=6
 15#16 [12,12,12,14,14,14,14,14,14,24,28,70,90,520,819] -> [12,12,12,14,90,520]; min=6
 15#17 [12,12,12,14,14,14,14,14,14,24,30,40,42,56,70] -> [12,12,12,14,30,40]; min=6
 15#18 [12,12,12,14,14,14,14,14,14,24,30,56,70,231,440] -> [12,12,12,14,30,231]; min=6
 15#19 [12,12,12,14,14,14,14,14,14,24,30,56,70,273,520] -> [12,12,12,14,30,273]; min=6
 15#20 [12,12,12,14,14,14,14,14,14,24,36,56,60,63,70] -> [12,12,12,14,24,60,63]; min=7
 15#21 [12,12,12,14,14,14,14,14,14,24,56,70,90,440,693] -> [12,12,12,14,90,440]; min=6
 15#22 [12,12,12,14,14,14,14,14,14,24,56,70,90,520,819] -> [12,12,12,14,90,520]; min=6
 15#23 [12,12,12,14,14,14,14,14,14,24,28,30,70,72,440,693] -> [12,12,12,14,30,440]; min=6
 15#24 [12,12,12,14,14,14,14,14,14,24,28,30,70,72,520,819] -> [12,12,12,14,30,520]; min=6
 15#25 [12,12,12,14,14,14,14,14,14,30,56,70,72,440,693] -> [12,12,12,14,30,440]; min=6
 15#26 [12,12,12,14,14,14,14,14,14,30,56,70,72,520,819] -> [12,12,12,14,30,520]; min=6
 15#27 [12,12,14,14,14,14,14,14,14,24,28,30,36,70,440,693] -> [12,12,14,24,30,693]; min=6
 15#28 [12,12,14,14,14,14,14,14,14,24,28,30,36,70,520,819] -> [12,12,14,24,30,819]; min=6
 15#29 [12,12,14,14,14,14,14,14,14,24,30,36,56,70,440,693] -> [12,12,14,24,30,693]; min=6
 15#30 [12,12,14,14,14,14,14,14,14,24,30,36,56,70,520,819] -> [12,12,14,24,30,819]; min=6
 16#01 [10,10,12,12,15,15,15,15,15,24,30,36,40,45,70,84] -> [10,12,12,15,24,36,84]; min=7
 16#02 [10,10,12,15,15,15,15,15,15,24,30,36,40,45,70,84] -> [10,10,12,15,15,15,15,70]; min=9
 16#03 [10,12,12,12,15,15,15,15,15,15,30,40,45,70,72,84] -> [10,12,12,12,15,72,84]; min=7
 16#04 [10,12,12,12,15,15,15,15,15,24,30,36,40,45,70,84] -> [10,12,12,12,15,24,36]; min=7
 16#05 [10,12,12,15,15,15,15,15,15,15,24,30,36,40,45] -> [10,12,12,15,15,15,15,24]; min=9
 16#06 [10,12,12,15,15,15,15,15,15,15,36,45,60,70,84] -> [10,12,12,15,15,15,15,36]; min=9
 16#07 [10,12,12,15,15,15,15,15,15,15,20,30,36,45,70,84] -> [10,12,12,15,15,15,15,36]; min=9
 16#08 [10,12,12,15,15,15,15,15,15,15,24,36,40,45,70,84] -> [10,12,12,15,24,36,84]; min=7
 16#09 [10,12,12,15,15,15,15,15,15,15,30,36,40,45,70,168] -> [10,12,12,15,15,15,15,36]; min=9
 16#10 [10,12,12,15,15,15,15,15,15,15,30,40,45,70,72,84] -> [10,12,12,15,15,15,15,72]; min=9
 16#11 [10,12,12,15,15,15,15,15,15,24,30,36,40,45,70,84] -> [10,12,12,15,24,36,84]; min=7
 16#12 [10,12,15,15,15,15,15,15,15,15,24,30,36,40,45] -> [10,12,15,15,15,15,15,24,36]; min=9
 16#13 [10,12,15,15,15,15,15,15,15,15,24,30,40,70,84] -> [10,12,15,15,15,15,15,24,84]; min=9
 16#14 [10,12,15,15,15,15,15,15,15,15,20,30,36,45,70,84] -> [10,12,15,15,15,15,15,36,84]; min=9
 16#15 [10,12,15,15,15,15,15,15,15,15,24,30,40,45,70,126] -> [10,12,15,15,15,15,15,126]; min=8
 16#16 [10,12,15,15,15,15,15,15,15,15,24,30,40,45,70,252] -> [10,12,15,15,15,15,15,24,252]; min=9
 16#17 [10,12,15,15,15,15,15,15,15,15,24,36,40,45,70,84] -> [10,12,15,15,15,15,15,24,36]; min=9
 16#18 [10,12,15,15,15,15,15,15,15,15,30,36,40,45,70,168] -> [10,12,15,15,15,15,15,36,168]; min=9
 16#19 [10,12,15,15,15,15,15,15,15,15,30,40,45,70,72,84] -> [10,12,15,15,15,15,15,72,84]; min=9
 16#20 [10,12,15,15,15,15,15,15,15,15,24,30,36,40,45,70,84] -> [10,12,15,15,15,15,15,24,36]; min=9
 16#21 [10,15,15,15,15,15,15,15,15,15,20,30,36,45,70,84] -> [10,15,15,15,15,15,15,15,36,70]; min=10
 16#22 [10,15,15,15,15,15,15,15,15,15,30,36,40,45,70,168] -> [10,15,15,15,15,15,15,15,36,70]; min=10
 16#23 [10,15,15,15,15,15,15,15,15,15,30,40,45,70,72,84] -> [10,15,15,15,15,15,15,15,70,72]; min=10
 16#24 [10,15,15,15,15,15,15,15,15,15,24,30,36,40,45,70,84] -> [10,15,15,15,15,15,15,24,36,84]; min=9
 16#25 [12,12,12,12,12,15,15,15,15,15,30,40,45,70,72,84] -> [12,12,12,12,12,15,70]; min=7
 16#26 [12,12,12,12,15,15,15,15,15,15,24,30,36,40,45] -> [12,12,12,12,15,15,15,15,40]; min=10
 16#27 [12,12,12,12,15,15,15,15,15,15,15,36,45,60,70,84] -> [12,12,12,12,15,36,70]; min=7
 16#28 [12,12,12,12,15,15,15,15,15,15,20,30,36,45,70,84] -> [12,12,12,12,15,36,70]; min=7
 16#29 [12,12,12,12,15,15,15,15,15,15,24,36,40,45,70,84] -> [12,12,12,12,15,24,70]; min=7
 16#30 [12,12,12,12,15,15,15,15,15,15,30,36,40,45,70,168] -> [12,12,12,12,15,36,70]; min=7
 16#31 [12,12,12,12,15,15,15,15,15,15,30,40,45,70,72,84] -> [12,12,12,12,15,70,72]; min=7
 16#32 [12,12,12,12,15,15,15,15,15,15,24,30,36,40,45,70,84] -> [12,12,12,12,15,24,70]; min=7
 16#33 [12,12,12,15,15,15,15,15,15,15,15,15,30,40,45,72] -> [12,15,15,15,15,15,15,15,15,40]; min=11
 16#34 [12,12,12,15,15,15,15,15,15,15,15,24,30,36,40,45] -> [12,12,12,15,15,15,15,15,36,40]; min=10
 16#35 [12,12,12,15,15,15,15,15,15,15,15,24,30,40,70,84] -> [12,12,12,15,15,15,15,15,70]; min=9
 16#36 [12,12,12,15,15,15,15,15,15,15,15,30,36,45,70,84] -> [12,12,12,15,15,15,15,15,70]; min=9
 16#37 [12,12,12,15,15,15,15,15,15,15,15,36,45,60,70,84] -> [12,12,12,15,15,15,15,15,70]; min=9
 16#38 [12,12,12,15,15,15,15,15,15,15,15,40,45,70,72,84] -> [12,12,12,15,15,15,15,15,70]; min=9
 16#39 [12,12,12,15,15,15,15,15,15,15,20,30,36,45,70,84] -> [12,12,12,15,15,15,15,15,70]; min=9
 16#40 [12,12,12,15,15,15,15,15,15,15,24,30,40,45,70,126] -> [12,12,12,15,40,70,126]; min=7
 16#41 [12,12,12,15,15,15,15,15,15,15,24,30,40,45,70,252] -> [12,12,12,15,15,15,15,15,70]; min=9
 16#42 [12,12,12,15,15,15,15,15,15,15,24,36,40,45,70,84] -> [12,12,12,15,24,36,70]; min=7
 16#43 [12,12,12,15,15,15,15,15,15,15,30,36,40,45,70,168] -> [12,12,12,15,15,15,15,15,70]; min=9
 16#44 [12,12,12,15,15,15,15,15,15,15,30,40,45,70,72,84] -> [12,12,12,15,15,15,15,15,70]; min=9
 16#45 [12,12,12,15,15,15,15,15,15,15,24,30,36,40,45,70,84] -> [12,12,12,15,24,36,70]; min=7
 16#46 [12,12,15,15,15,15,15,15,15,15,20,36,45,70,84] -> [12,12,15,15,15,15,15,36,70]; min=9

16#47 [12,12,15,15,15,15,15,15,15,15,15,24,30,36,40,45] -> [12,15,15,15,15,15,15,15,15,15,40]; min=11
 16#48 [12,12,15,15,15,15,15,15,15,15,15,24,30,40,70,84] -> [12,12,15,15,15,15,15,24,70]; min=9
 16#49 [12,12,15,15,15,15,15,15,15,15,15,30,36,45,70,84] -> [12,12,15,15,15,15,15,36,70]; min=9
 16#50 [12,12,15,15,15,15,15,15,15,15,15,36,40,45,70,168] -> [12,12,15,15,15,15,15,36,70]; min=9
 16#51 [12,12,15,15,15,15,15,15,15,15,15,36,45,60,70,84] -> [12,12,15,15,15,15,15,36,70]; min=9
 16#52 [12,12,15,15,15,15,15,15,15,15,15,40,45,70,72,84] -> [12,12,15,15,15,15,15,70,72]; min=9
 16#53 [12,12,15,15,15,15,15,15,15,15,20,30,36,45,70,84] -> [12,12,15,15,15,15,15,36,70]; min=9
 16#54 [12,12,15,15,15,15,15,15,15,15,24,30,40,45,70,126] -> [12,12,15,15,15,15,15,24,70]; min=9
 16#55 [12,12,15,15,15,15,15,15,15,15,24,30,40,45,70,252] -> [12,12,15,15,15,15,15,24,70]; min=9
 16#56 [12,12,15,15,15,15,15,15,15,15,24,36,40,45,70,84] -> [12,12,15,15,15,15,15,24,70]; min=9
 16#57 [12,12,15,15,15,15,15,15,15,15,30,36,40,45,70,168] -> [12,12,15,15,15,15,15,36,70]; min=9
 16#58 [12,12,15,15,15,15,15,15,15,15,30,40,45,70,72,84] -> [12,12,15,15,15,15,15,70,72]; min=9
 16#59 [12,12,15,15,15,15,15,15,15,24,30,36,40,42,45,70] -> [12,12,15,36,40,42,70]; min=7
 16#60 [12,12,15,15,15,15,15,15,15,24,30,36,40,45,70,84] -> [12,12,15,15,15,15,15,24,70]; min=9
 16#61 [12,15,15,15,15,15,15,15,15,15,15,24,30,40,70,84] -> [12,15,15,15,15,15,15,15,15,40]; min=11
 16#62 [12,15,15,15,15,15,15,15,15,15,20,30,36,45,70,84] -> [12,15,15,15,15,15,15,15,15,20]; min=11
 16#63 [12,15,15,15,15,15,15,15,15,15,24,30,40,45,70,126] -> [12,15,15,15,15,15,15,15,15,40];
 ↪ min=11
 16#64 [12,15,15,15,15,15,15,15,15,15,24,30,40,45,70,252] -> [12,15,15,15,15,15,15,15,15,40];
 ↪ min=11
 16#65 [12,15,15,15,15,15,15,15,15,15,24,36,40,45,70,84] -> [12,15,15,15,15,15,24,36,70]; min=9
 16#66 [12,15,15,15,15,15,15,15,15,15,30,36,40,45,70,168] -> [12,15,15,15,15,15,15,15,15,40];
 ↪ min=11
 16#67 [12,15,15,15,15,15,15,15,15,15,30,40,45,70,72,84] -> [12,15,15,15,15,15,15,15,15,40]; min=11
 16#68 [12,15,15,15,15,15,15,15,15,18,24,30,40,45,70,84] -> [12,15,15,15,15,15,18,70]; min=8
 16#69 [12,15,15,15,15,15,15,15,15,24,30,36,40,42,45,70] -> [12,15,15,15,15,15,24,36,70]; min=9
 16#70 [12,15,15,15,15,15,15,15,15,24,30,36,40,45,70,84] -> [12,15,15,15,15,15,24,36,70]; min=9
 16#71 [15,15,15,15,15,15,15,15,15,24,30,36,40,45,70,84] -> [15,15,15,15,15,15,15,15,24,40]; min=11
 16#72 [15,15,15,15,15,15,15,15,15,30,36,45,70,110,260,9009] -> [15,15,15,15,15,15,15,36,70,110];
 ↪ min=10
 16#73 [15,15,15,15,15,15,15,15,15,30,36,45,70,130,220,9009] -> [15,15,15,15,15,15,15,36,70,130];
 ↪ min=10
 16#74 [15,15,15,15,15,15,15,15,15,30,36,45,110,130,140,9009] -> [15,15,15,15,15,15,15,36,110,130];
 ↪ min=10
 16#75 [15,15,15,15,15,15,15,15,15,30,45,70,72,110,520,9009] -> [15,15,15,15,15,15,15,70,72,110];
 ↪ min=10
 16#76 [15,15,15,15,15,15,15,15,15,30,45,70,72,130,440,9009] -> [15,15,15,15,15,15,15,70,72,130];
 ↪ min=10
 16#77 [15,15,15,15,15,15,15,15,15,30,45,72,110,130,280,9009] -> [15,15,15,15,15,15,15,72,110,130];
 ↪ min=10